



Tecnológico Nacional de México

Instituto Tecnológico de Chiná

Nombre de la carrera:

Ingeniería en Informática.

Nombre de la asignatura:

Programación cliente - servidor.

Semestre y grupo:

7AI

Actividad:

Investigación de la Unidad 1.

Nombre del alumno:

Aaron Jesús Eloy Aguilar León.

Nombre del docente:

Jenner Noe Che Méndez.

Lugar y fecha:

San Francisco de Campeche, Campeche a 03 de Octubre de 2026.

Índice

Unidad 1 Arquitecturas cliente – servidor contemporáneas y diseño API-first.	3
Subtema 1.1 Evolución del modelo cliente – servidor: proceso cliente, proceso servidor, servicio, middleware y sistema distribuido.	3
Subtema 1.2 Funciones de redes publicas aplicados: DNS, IP, puertos, TCP, HTTP/ HTTPS, petición, respuesta, latencia y TLS.	7
Subtema 1.3 Arquitecturas de dos, tres y N capas; monolito modular, microservicios, orientada a eventos, serverless y Backend as a Service (BaaS).	11
Subtema 1.4 Estilos y protocolos: REST, RPC / gRPC, WebSocket, Server-Sent Events, webhooks y mensajería; criterios de selección.	15
Subtema 1.5 Separación de responsabilidades, escalabilidad, disponibilidad, consistencia, costo, complejidad y deuda operativa.	18
Subtema 1.6 Diseño API-first, diagramas C4 simplificados, contrato inicial y registros de decisión arquitectónica (ADR).	21
Referencias.	24

Unidad 1 Arquitecturas cliente – servidor contemporáneas y diseño API-first.

Subtema 1.1 Evolución del modelo cliente – servidor: proceso cliente, proceso servidor, servicio, middleware y sistema distribuido.

Definición y Componentes Fundamentales.

El modelo cliente-servidor es un patrón de arquitectura de software que distribuye las tareas y la carga de trabajo entre los proveedores de un recurso o servicio (servidores) y los demandantes de dicho servicio (clientes).

Proceso Cliente.

Es la entidad o programa que se ejecuta en el dispositivo del usuario final e inicia activamente las solicitudes de servicio hacia el backend.

Funciones clave: Captura de datos de entrada del usuario, gestión de la interfaz de usuario (UI), validación básica del lado del cliente (frontend validation) y renderizado de respuestas.

Tipos de cliente:

- Cliente Pesado (Fat/Thick Client): Ejecuta la mayor parte de la lógica de negocio y presentación localmente; solo requiere al servidor para almacenar datos o ejecutar consultas de base de datos.
- Cliente Ligero (Thin Client): Depende casi por completo del servidor para el procesamiento de la lógica de negocio; solo se encarga de mostrar la interfaz (ejemplo: un navegador web que renderiza HTML/CSS).
- Cliente Híbrido / Rico (Rich Internet Application - RIA): Combina la capacidad de procesamiento local mediante JavaScript/Frameworks (como React, Angular, Vue) con el consumo de APIs en el servidor.

Proceso Servidor.

Es el programa o conjunto de aplicaciones en ejecución que permanece en un estado de escucha activa en un puerto de red específico, esperando las peticiones entrantes de uno o varios clientes.

Funciones clave: Recepción de peticiones, autenticación y autorización del usuario, ejecución de reglas de negocio, interacción con la capa de persistencia (bases de datos) y generación de una respuesta con códigos de estado estandarizados.

Propiedades: Opera en modo sin estado (stateless) o con estado (stateful), gestiona múltiples conexiones simultáneas mediante hilos (multithreading) o eventos asíncronos, y garantiza la integridad de las transacciones.

Servicio.

Es la funcionalidad técnica o de negocio abstraída, bien definida y reutilizable que expone una interfaz estandarizada para ser consumida por los clientes.

Características:

- Encapsulamiento: Oculta los detalles de implementación del código y la base de datos interna.
- Desacoplamiento: Permite modificar la lógica interna del servicio sin afectar a los clientes que consumen la interfaz siempre que se mantenga el mismo contrato.
- Reutilización: Una misma lógica (por ejemplo, el cálculo de impuestos o la autenticación) puede ser consumida por la aplicación web, la app móvil y sistemas de terceros.

2.- Capas de Integración y Extensión.

Middleware.

Es el software intermedio que actúa como puente de comunicación, integración y abstracción entre aplicaciones, sistemas operativos, bases de datos o redes heterogéneas.

Funciones principales:

- Enrutamiento y Transformación: Traduce formatos de datos entre cliente y servidor (ejemplo: conversión de JSON a llamadas de procedimientos).
- Gestión de Seguridad: Intercepta peticiones para validar credenciales, tokens (JWT) y aplicar políticas de acceso.
- Manejo de Transacciones: Asegura la consistencia en operaciones distribuidas (protocolo Two-Phase Commit).
- Balanceo de Carga y Colas: Distribuye el tráfico entrante entre múltiples instancias de servidores para evitar saturaciones.

Sistema Distribuido.

Es un conjunto de computadoras independientes (nodos) conectadas a través de una red que coordinan sus acciones pasando mensajes, presentándose ante el usuario final como un único sistema coherente y centralizado.

Objetivos principales:

- Tolerancia a fallos: Si un nodo cae, otros componentes asumen la carga sin interrumpir el servicio total.
- Escalabilidad Horizontal: Permite incrementar la capacidad de procesamiento añadiendo más nodos/servidores a la red.
- Transparencia: Oculta al usuario la ubicación física de las máquinas, el acceso a los datos y la replicación de los recursos.

3.- Evolución Histórica del Modelo.

Era 1: Arquitectura Centralizada (Mainframes y Terminales Tontas) - Años 60 a 70.

Estructura: Cómputo, memoria, lógica y almacenamiento concentrados al 100% en una supercomputadora central (Mainframe).

Interacción: El usuario interactúa desde terminales que solo enviaban entradas de teclado y mostraban texto en pantalla.

Problema: Monopolio operativo, costos de infraestructura extremadamente elevados y nula flexibilidad visual.

Era 2: Cliente-Servidor de 2 Capas (Two-Tier) - Años 80 a 90.

Estructura: La llegada de las computadoras personales (PC) permite dividir el sistema en dos partes: Cliente Pesado (interfaz + lógica de negocio) y Servidor de Base de Datos (almacenamiento).

Interacción: Las PC de escritorio ejecutan software compilado localmente y se conectan mediante drivers (ODBC/JDBC) a la base de datos de la red local (LAN).

Problema: Problemas de mantenimiento (DLL Hell), baja seguridad (acceso directo a la base de datos desde el cliente) e imposibilidad de escalar fuera de la red local.

Era 3: Arquitectura Multicapa (Three-Tier / N-Tier) - Años 90 a 2000.

Estructura: Impulsada por la adopción masiva de la Web. Se introduce una tercera capa intermedia:

- Capa de Presentación: Navegador web (Cliente ligero).
- Capa de Aplicación: Servidor web/aplicaciones que procesa la lógica del negocio.
- Capa de Datos: Servidor de base de datos relacional.

Ventaja: Desacoplamiento de la lógica del negocio, despliegues centralizados y mayor seguridad.

Era 4: Sistemas Distribuidos, Microservicios y Nube - 2010 a la actualidad.

Estructura: La aplicación monolítica se divide en múltiples servicios pequeños e independientes (Microservicios) desplegados en la nube o en contenedores.

Interacción: Clientes móviles y aplicaciones de una sola página (SPA) interactúan con un API Gateway (Middleware) mediante contratos API REST, gRPC o WebSockets.

Impacto: Alta disponibilidad, escalabilidad elástica automatizada y delegación de infraestructura a plataformas en la nube (BaaS/Serverless).

Subtema 1.2 Funciones de redes publicas aplicados: DNS, IP, puertos, TCP, HTTP/ HTTPS, petición, respuesta, latencia y TLS.

1.- Identificación y Enrutamiento en Red (DNS e IP).

DNS (Domain Name System).

Es un sistema de nombres distribuido y jerárquico que actúa como el directorio telefónico de Internet.

Función: Traduce nombres de dominio legibles para los humanos (como api.biblioteca.com) en direcciones IP numéricas procesables por las máquinas (192.0.2.1).

Mecanismo: Sigue un flujo jerárquico de resolución: consulta en caché local - Servidor - DNS Resolver - Servidor Raíz(.) - Servidor TLD (.com) - Servidor Autoritativo.

IP (Internet Protocol).

Es el protocolo principal de la capa de red que se encarga del direccionamiento y enrutamiento de paquetes de datos a través de los límites de las redes.

Versiones:

- IPv4: Direcciones de 32 bits representadas en formato decimal con puntos (ej. 192.168.1.1).
- IPv6: Direcciones de 128 bits expresadas en formato hexadecimal separadas por dos puntos (ej. 2001:0db8:85a3::8a2e: 0370:7334).

Propiedad: Es un protocolo no orientado a conexión (connectionless) y de "mejor esfuerzo" (best-effort), lo que significa que no garantiza la entrega ni el orden de los paquetes (tarea delegada a la capa de transporte).

2.- Multiplexación y Capa de Transporte Confiable (Puertos y TCP).

Puertos.

Son puntos finales lógicos asignados por el sistema operativo (rango de 0 a 65535) que permiten la multiplexación en una misma interfaz de red.

Propósito: Permiten que un mismo servidor atienda diferentes tipos de tráfico de forma simultánea distinguiendo la aplicación de destino (ej. Puerto 80 para HTTP, 443 para HTTPS, 5432 para PostgreSQL).

TCP (Transmission Control Protocol).

Protocolo de la capa de transporte orientado a conexión que garantiza que los datos se entreguen de forma completa, ordenada y sin errores.

Handshake de 3 Vías (Three-Way Handshake):

- SYN: El cliente envía un segmento con el flag SYN para solicitar la conexión.
- SYN-ACK: El servidor responde confirmando la recepción y solicitando la conexión de vuelta.
- ACK: El cliente confirma la respuesta y se establece el canal de comunicación.

Mecanismos de Control: Implementa control de flujo (sliding window) y control de congestión para adaptar el envío a la capacidad de la red y del receptor.

3.- Capa de Aplicación y Modelo Petición-Respuesta (HTTP/HTTPS, Request y Response).

HTTP / HTTPS.

Protocolo de la capa de aplicación sin estado (stateless) utilizado para la transferencia de recursos estructurados en la web. HTTPS es la versión segura que encapsula las peticiones HTTP dentro de un túnel cifrado por el protocolo TLS.

Petición HTTP (Request).

Mensaje estandarizado que el cliente emite hacia el servidor solicitando la ejecución de una acción específica.

Se compone de:

- Línea de Solicitud: Método HTTP (GET, POST, PUT, DELETE, PATCH), URI de destino y versión del protocolo.
- Encabezados (Headers): Metadatos clave-valor que envían contexto (Content-Type, Authorization, Accept, User-Agent).
- Cuerpo (Body): Carga útil con los datos formateados (comúnmente JSON o XML) enviados al servidor (utilizado principalmente en POST, PUT y PATCH).

Respuesta HTTP (Response).

Mensaje enviado por el servidor de vuelta al cliente con el resultado del procesamiento.

Se compone de:

Línea de Estado: Código de estado numérico y mensaje asociado:

- 1xx: Informativos.
- 2xx: Éxito (200 OK, 201 Created, 204 No Content).
- 3xx: Redirección (301 Moved Permanently, 304 Not Modified).
- 4xx: Errores del cliente (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found).
- 5xx: Errores del servidor (500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable).

Encabezados: Metadatos sobre la respuesta (Set-Cookie, Cache-Control, Access-Control-Allow-Origin).

Cuerpo (Body): Los datos solicitados o los detalles del error procesado.

4.- Rendimiento y Cifrado en Red (Latencia y TLS).

Latencia.

Tiempo total que le toma a un paquete de datos viajar desde el origen hasta el destino y retornar (tiempo de ida y vuelta o Round Trip Time - RTT).

Factores determinantes: Distancia física entre cliente y servidor, número de saltos de red (routers), congestión de la red y tiempo de procesamiento en el servidor.

TLS (Transport Layer Security).

Protocolo criptográfico que provee privacidad, autenticidad e integridad en las comunicaciones sobre la red.

Proceso de Handshake TLS:

Negociación de Algoritmos: Cliente y servidor acuerdan la suite criptográfica.

Autenticación del Servidor: El servidor presenta un certificado digital (firmado por una Autoridad Certificadora) que contiene su clave pública.

Intercambio de Claves: Mediante criptografía asimétrica, se genera y negocia una clave simétrica temporal.

Cifrado Simétrico: Todo el tráfico HTTP posterior se cifra usando la clave simétrica negociada, garantizando alta velocidad y confidencialidad.

Subtema 1.3 Arquitecturas de dos, tres y N capas; monolito modular, microservicios, orientada a eventos, serverless y Backend as a Service (BaaS).

1.- Arquitecturas Basadas en Capas (Layered Architectures).

La división en capas agrupa el código según su responsabilidad lógica dentro del sistema, permitiendo un desarrollo estructurado y la separación de preocupaciones (Separation of Concerns).

Arquitectura de Dos Capas (Two-Tier).

Es el modelo cliente-servidor tradicional de los años 80 y 90.

Estructura:

- Capa Cliente (Pesado): Contiene la interfaz de usuario (UI) y toda la lógica de negocio.
- Capa de Datos: Un servidor central de base de datos relacional.

Características: El cliente se conecta directamente a la base de datos mediante drivers o consultas SQL directas.

Desventajas: Poca escalabilidad, alto riesgo de seguridad (credenciales expuestas en el cliente) y dificultad extrema para mantener o actualizar el software en los clientes.

Arquitectura de Tres Capas (Three-Tier).

Surgió con el auge de la Web para desacoplar el cliente de la base de datos introduciendo un servidor intermedio.

Estructura:

- Capa de Presentación (UI): Renderiza la interfaz en el cliente (navegador web o app).
- Capa de Lógica de Negocio (Backend): Servidor de aplicaciones que procesa las reglas del sistema y validaciones

- Capa de Datos (Persistencia): Gestor de base de datos (DBMS).

Características: Mayor seguridad y facilidad de mantenimiento, ya que las reglas de negocio están centralizadas en la capa intermedia.

Arquitectura de N Capas (N-Tier).

Evolución de la arquitectura de tres capas donde la capa de negocio o de datos se subdivide en múltiples subcapas especializadas.

Estructura típica: Presentación - Controladores/API -Servicios de Negocio - Capa de Acceso de Datos (ORM/Repositorios) - Base de datos.

Ventajas: Alta modularidad, desacoplamiento estricto y facilidad para realizar pruebas unitarias en cada capa.

2.- Estilos de Organización Monolítica y Distribuida.

Monolito Modular.

Es un enfoque arquitectónico donde todo el sistema se empaqueta y despliega como una única unidad ejecutable, pero su código fuente interno está estrictamente dividido en módulos independientes y desasociados.

Características: Cada módulo encapsula su propia lógica y modelo de dominio, comunicándose mediante interfaces o llamadas a funciones internas claras.

Ventajas: Combina la simplicidad de despliegue y pruebas de un monolito con la mantenibilidad y límites limpios de una arquitectura modular.

Microservicios.

Estilo arquitectónico que estructura una aplicación como una colección de servicios pequeños, independientes y autónomos.

Características:

- Cada servicio responde a un dominio de negocio específico (Bounded Context).

- Tienen sus propias bases de datos independientes (Database per Service).
- Se comunican a través de la red mediante protocolos ligeros (REST, gRPC, mensajería).
- Se despliegan de forma totalmente independiente mediante contenedores (Docker/Kubernetes).

Ventajas: Escalabilidad granular, tolerancia a fallos aislada y libertad tecnológica por servicio.

3.- Arquitecturas Orientadas a Eventos y Modelos Cloud Native.

Arquitectura Orientada a Eventos (EDA - Event-Driven Architecture).

Es un patrón de diseño donde el flujo del sistema está guiado por la producción, detección y consumo de eventos.

Componentes:

- Emisor (Publisher): Genera una notificación cuando ocurre un cambio de estado (ej. PedidoCreado).
- Intermediario (Broker): Plataforma que gestiona y distribuye eventos (ej. RabbitMQ, Apache Kafka).
- Consumidor (Subscriber): Reacciona de forma asíncrona al evento para ejecutar sus tareas.

Ventajas: Desacoplamiento temporal y espacial completo entre componentes y alto rendimiento para procesamiento asíncrono.

Serverless (Computación sin Servidor / FaaS).

Modelo de ejecución en la nube donde el desarrollador escribe código en forma de funciones puras (Function as a Service) y el proveedor gestiona toda la infraestructura.

Características:

- Petición/Respuesta por eventos: La función se ejecuta solo cuando un evento la activa.
- Escalado a cero: Si no hay peticiones, no hay instancias activas ni costo financiero.

Ejemplos: AWS Lambda, Google Cloud Functions.

Backend as a Service (BaaS).

Modelo donde se delega la gestión del backend completo a una plataforma en la nube lista para usar.

Servicios provistos: Autenticación de usuarios, bases de datos en tiempo real, almacenamiento de archivos y funciones serverless expuestas directamente mediante SDKs o APIs.

Casos de uso: Desarrollo acelerado de aplicaciones web y móviles sin necesidad de construir un servidor backend desde cero.

Ejemplos: Supabase, Firebase.

Subtema 1.4 Estilos y protocolos: REST, RPC / gRPC, WebSocket, Server-Sent Events, webhooks y mensajería; criterios de selección.

1.- Estilos y Protocolos de Comunicación.

REST (Representational State Transfer).

Estilo arquitectónico orientado a recursos, sin estado (stateless), que utiliza las abstracciones estándar del protocolo HTTP.

Principios: Los recursos se identifican mediante URIs unívocas (ej. /api/v1/libros/42) y se manipulan utilizando verbos HTTP estándar (GET, POST, PUT, DELETE, PATCH).

Usa formatos livianos como JSON para el intercambio de datos.

Casos de uso: APIs públicas, integración de aplicaciones web y móviles tradicionales donde se requiera sencillez y alta interoperabilidad.

RPC / gRPC (Remote Procedure Call).

Estilo de comunicación orientado a acciones donde un cliente invoca directamente un método o procedimiento en un servidor remoto como si fuera una llamada local.

gRPC (Google RPC): Implementación de alto rendimiento que utiliza HTTP/2 como transporte y Protocol Buffers (Protobuf) como formato de serialización binaria en lugar de JSON.

Ventajas: Extremadamente rápido, bajo consumo de ancho de banda, soporta streaming bidireccional y tipado estricto mediante archivos de definición (.proto).

Casos de uso: Comunicación interna inter-servicios en arquitecturas de microservicios e integración de baja latencia.

WebSocket.

Protocolo de comunicación de la capa de aplicación que proporciona un canal de transmisión dúplex completo (bidireccional) y persistente sobre una única conexión TCP.

Mecanismo: Comienza como una petición HTTP estándar que realiza un "apretón de manos" (handshake) para elevar el protocolo (Upgrade: websocket).

Una vez establecido, ambas partes pueden enviarse mensajes en tiempo real sin el encabezado repetitivo de HTTP.

Casos de uso: Aplicaciones de chat en vivo, editores colaborativos en tiempo real, juegos multijugador y paneles de control financieros.

Server-Sent Events (SSE) Mecanismo estandarizado en HTML5 donde el cliente abre una conexión HTTP persistente y el servidor le transmite datos en tiempo real de forma unidireccional (servidor - cliente).

Mecanismo: Utiliza el tipo de contenido text/event-stream, a diferencia de WebSocket, la reconexión es automática y nativa dentro del protocolo HTTP estándar.

Casos de uso: Notificaciones en tiempo real, feed de noticias, monitoreo de métricas e interfaces de respuesta generativa (como respuestas de modelos de IA).

Webhooks.

Patrón de comunicación asíncrona basado en eventos donde un servidor origen envía una notificación mediante una petición HTTP POST automática a una URL expuesta por el servidor cliente cuando ocurre un evento específico.

Mecanismo: Funciona bajo el principio de "no me llames, yo te llamo" (push-based), eliminando la necesidad de que el cliente realice peticiones periódicas de consulta (polling).

Casos de uso: Procesamiento de pagos (ej. pasarelas como Stripe/PayPal), pipelines de integración continua (CI/CD) y notificaciones de repositorios (GitHub).

Mensajería (Message Brokers).

Patrón de integración asíncrona mediante un intermediario de mensajes (message broker) que gestiona la entrega de información desacoplando el emisor del receptor.

Patrones principales:

- Punto a Punto (Colas): Un mensaje es procesado por un único consumidor (ej. RabbitMQ, SQS).
- Publicación/Suscripción (Tópicos): Un evento se emite a un tópico y es recibido por múltiples suscriptores independientes (ej. Apache Kafka, Google Pub/Sub).

Casos de uso: Procesamiento de tareas pesadas en segundo plano, procesamiento de streams masivos de datos y coordinación en sistemas distribuidos.

Subtema 1.5 Separación de responsabilidades, escalabilidad, disponibilidad, consistencia, costo, complejidad y deuda operativa.

1.- Atributos de Calidad y Criterios Arquitectónicos.

Separación de Responsabilidades (Separation of Concerns).

Principio de diseño que establece que un sistema de software debe dividirse en partes o componentes distintos, donde cada uno aborda una única responsabilidad o funcionalidad específica.

Impacto: Evita la creación de "clases o módulos Dios" (God Objects), facilita el mantenimiento, permite probar componentes de forma aislada y acelera el desarrollo paralelo entre equipos.

Escalabilidad.

Capacidad de un sistema para adaptar su rendimiento y manejar un volumen creciente de trabajo (peticiones, usuarios concurrentes o volumen de datos) mediante la adición de recursos informáticos.

Tipos:

- Escalabilidad Vertical (Scale-Up): Aumentar la capacidad física del nodo existente (incrementar CPU, memoria RAM o almacenamiento).
- Escalabilidad Horizontal (Scale-Out): Añadir más nodos o instancias al sistema distribuido y repartir la carga mediante un balanceador de tráfico.

Disponibilidad.

Porcentaje de tiempo que un sistema o servicio se mantiene plenamente operativo y accesible para responder a las solicitudes válidas de los usuarios.

Métrica: Se mide habitualmente en los "nueves" de disponibilidad (ej. 99% o 3 nueves, lo que equivale a un máximo de ~8.76 horas de inactividad al año).

Estrategias: Redundancia de componentes, balanceo de carga y tolerancia a fallos (failover).

Consistencia.

Propiedad que garantiza que todos los nodos o clientes de un sistema distribuido observen exactamente la misma información o estado de los datos al mismo tiempo.

Modelos de Consistencia:

- Consistencia Fuerte (Strong Consistency): Tras una escritura, cualquier lectura posterior en cualquier nodo devolverá el valor recién escrito (común en bases de datos SQL).
- Consistencia Eventual (Eventual Consistency): Los nodos no se actualizan instantáneamente, pero se garantiza que, tras un periodo sin nuevas escrituras, todos convergerán al mismo estado (común en bases de datos NoSQL y sistemas distribuidos masivos).

2.- Factores de Compensación (Trade-offs) y Costos.

Costo.

Gasto financiero total asociado al ciclo de vida completo de la arquitectura.

Dimensiones:

- CapEx (Costo de Capital): Inversión inicial en infraestructura propia (servidores físicos, licencias permanentes).
- OpEx (Costo Operativo): Gastos recurrentes en servicios en la nube (AWS, GCP, Azure), ancho de banda y mantenimiento.

Complejidad.

Grado de dificultad en la estructura, desarrollo, depuración, despliegue y comprensión general del sistema.

Efecto: A mayor grado de distribución (por ejemplo, pasar de un monolito a cientos de microservicios), la complejidad de la red, la orquestación y el rastreo de errores (distributed tracing) aumentan de forma exponencial.

Deuda Operativa.

Carga técnica y de esfuerzo acumulado que surge al mantener soluciones temporales, configuraciones manuales, falta de automatización o infraestructuras sobre-diseñadas (over-engineering).

Impacto: Reduce la velocidad de entregas futuras, incrementa el riesgo de caídas en producción y requiere un porcentaje significativo del tiempo del equipo devops para "apagar incendios" en lugar de construir características de valor.

3.- El Teorema CAP (Fundamento de Compensación).

En el diseño de sistemas distribuidos, el Teorema CAP demuestra que es imposible garantizar simultáneamente más de dos de las siguientes tres propiedades:

- Consistencia (C): Todos los nodos leen los mismos datos al mismo tiempo.
- Disponibilidad (A): Toda petición no errónea recibe una respuesta no fallida.
- Tolerancia a Particiones (P): El sistema sigue funcionando a pesar de pérdidas o retrasos en la comunicación de red entre nodos.

Regla en redes reales: Puesto que las fallas de red son inevitables en un sistema distribuido (P se debe asumir), se debe elegir conscientemente entre Consistencia + Tolerancia a Particiones (CP) o Disponibilidad + Tolerancia a Particiones (AP).

Subtema 1.6 Diseño API-first, diagramas C4 simplificados, contrato inicial y registros de decisión arquitectónica (ADR).

1.- Metodología de Diseño y Especificación (API-First y Contrato Inicial).

Diseño API-First.

Enfoque de arquitectura y desarrollo que trata a las APIs como "ciudadanos de primera clase", consiste en diseñar, especificar y validar la interfaz de programación de aplicaciones antes de escribir el código del backend o del frontend.

Flujo de trabajo:

- Identificación de necesidades del dominio.
- Redacción y aprobación de la especificación técnica.
- Generación de mocks o servidores simulados para desarrollo paralelo.
- Implementación del código base.

Ventajas:

- Desarrollo en paralelo: Equipos de frontend y backend pueden trabajar simultáneamente usando servidores mock.
- Interoperabilidad: Garantiza interfaces consistentes y orientadas al consumidor.
- Reducción de costos de cambio: Es más rápido corregir un diseño en un documento de especificación que refactorizar código ya compilado.

Contrato Inicial (API Contract).

Documento formal e inmutable que define las reglas de interacción entre el cliente y el servidor.

Estándar: Se escribe comúnmente utilizando OpenAPI Specification (OAS) (anteriormente Swagger) en formatos YAML o JSON.

Elementos del contrato:

- Rutas (Endpoints) y métodos HTTP (GET /api/v1/libros, POST /api/v1/prestamos).
- Parámetros de entrada (de consulta, de ruta, encabezados).
- Estructura del cuerpo de la petición y de la respuesta (Schemas).
- Códigos de estado HTTP devueltos y formatos de respuesta ante errores.

2.- Modelado de Arquitectura: Diagramas C4 Simplificados.

El Modelo C4 es un enfoque jerárquico para diagramar la arquitectura de software utilizando diferentes niveles de abstracción (similar al zoom de un mapa).

Niveles de Abstracción.

Nivel 1: Contexto (System Context):

- Muestra el sistema en su totalidad como una "caja negra" y su relación con los usuarios finales y otros sistemas externos.

Nivel 2: Contenedores (Containers):

- Desglosa el sistema en sus bloques de construcción de alto nivel desplegables (ej. Aplicación Web SPA, App Móvil, API Gateway, Microservicio de Autenticación, Base de Datos PostgreSQL).

Nivel 3: Componentes (Components):

- Abre un contenedor individual para mostrar sus componentes internos y responsabilidades (ej. Controladores, Servicios de Negocio, Repositorios).

Nivel 4: Código (Code):

- Detalle a nivel de diagramas de clases o código (comúnmente omitido en diagramación simplificada).

3.- Documentación de Decisiones: Registros de Decisión Arquitectónica (ADR).

Un ADR (Architecture Decision Record) es un documento de texto simple y breve que captura una decisión técnica o de diseño significativa tomada en el proyecto, junto con su contexto y sus consecuencias.

Estructura Estándar de un ADR.

- Título: Nombre claro y numerado de la decisión (ej. ADR-003: Selección de Supabase como BaaS para el Backend).
- Estado: Estado actual de la decisión (Prouesto, Aceptado, Rechazado, Deprecado, Sustituido).
- Contexto: Problema técnico o de negocio que se necesita resolver y las restricciones del entorno.
- Decisión: La solución arquitectónica seleccionada y la justificación técnica.
- Consecuencias: Resultados positivos y negativos (trade-offs) derivados de adoptar la decisión.

Referencias.

Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2012). Sistemas distribuidos: Conceptos y diseño (5.^a ed.). Pearson Educación.

Kurose, J. F., & Ross, K. W. (2017). Redes de computadoras: Un enfoque descendente (7.^a ed.). Pearson Educación.

Richards, M., & Ford, N. (2021). Fundamentos de arquitectura de software: Un enfoque integral (1.^a ed. en español). O'Reilly Media / Marcombo.